

Integration testing locally and on GitLab CI using LocalStack

Cite as: Martin Paul Eve, "Integration testing locally and on GitLab CI using LocalStack", *eve.gd: Martin Paul Eve*, March 19, 2023, <https://doi.org/10.59348/jvnf9-ref68>.



LocalStack is a great cloud emulation layer. It lets you simulate interaction with AWS, which is great for writing integration tests.

However, I wanted a system that, when run locally, would spin up the LocalStack server and then destroy it when done. But when running the test on GitLab CI, it will use the “service” provision of their continuous integration system and connect to that.

To do this, first you need to setup your `.gitlab-ci.yml` file:

```

run_tests:
  stage: test
  artifacts:
    when: always
    paths:
      - allure_results
    expire_in: 1 day
  image:
    name: python:3.9
    entrypoint: ["/usr/bin/env"]
  variables:
    HOSTNAME_EXTERNAL: localstack
    DEFAULT_REGION: us-east-1
    AWS_ACCESS_KEY_ID: localkey
    AWS_SECRET_ACCESS_KEY: localsecret
  services:
    - name: localstack/localstack
      alias: localstack
  before_script:
    - pip install pytest pytest-mock pytest-asyncio allure-pytest
    - pip install -r $CI_PROJECT_DIR/crapiproxy/layer-python/requirements.txt
  script:
    - cd $CI_PROJECT_DIR/crapiproxy/tests
    - python -m pytest -v --alluredir=$CI_PROJECT_DIR/allure_results
    - cd -
    - ls
  allow_failure: true

```

To spin up the docker container when running locally, we need the [pytest-localstack module](#), so add that to your requirements.

Then, to achieve the magic, I have a pytest fixture that looks like this:

```

@pytest.fixture()
def setup_s3(self):
    sys.path.append("../..")
    sys.path.append("../src")
    sys.path.append("../src/plugins")

    from plugins.utils import aws_utils
    import settings

    # first check if localstack is running
    if not os.environ.get("GITLAB_CI"):
        import docker

        client = docker.from_env()

        with LocalstackSession(
            services=["s3"], docker_client=client
        ) as session:
            self._aws_connector = aws_utils.AWSConnector(
                bucket=settings.BUCKET,
                endpoint_url=session.endpoint_url("s3"),
            )
            res = self.stage_two_setup()
            yield res
    else:
        # change the endpoint URL to use GitLab CLI version
        self._aws_connector = aws_utils.AWSConnector(
            bucket=settings.BUCKET,
            endpoint_url="http://localstack:4566",
        )

        res = self.stage_two_setup()
        yield res

```

Essentially, we check for the existence of the GITLAB_CI variable to ascertain where we are running. When we are running on GitLab CI, we set the endpoint_url for all of our AWS calls to http://localstack:4566. ([AWSConnector is a custom class that I wrote to handle this.](#))

However, if we are running locally, we start a LocalStack session. You have to do this this way, using the context manager, rather than the pytest fixture, because the fixture version runs automatically regardless of the location.